Programming and Data Structures Laboratory (CS19003)
Assignment 7 (Section 6): Dynamic Memory Allocation, Structure
Department of CSE, IIT Kharagpur

25th March, 2026

General instructions to be followed strictly:

- Name your file as <roll_no>_<assignment_no>_<question_no>. For example, if your roll number is `14CS10001` and you are submitting assignment 3 question 1, then name your file as `14CS10001_3_1.c`
- Write your name, roll number, and assignment number at the beginning of your program.
- Use proper indentation in your code and comment. Marks may be deducted without proper indentation and comments.
- Submitting the **correct file** is your responsibility, any request of re-submission after the lab will **never** be entertained.
- **Zero tolerance** policy will be stricly followed for **plagiarism**. The marks will be put straight zero if such cases are identified.

Question-1 (10 points)

Write a C program that uses dynamic memory allocation to store and process a list of integers. The program should first ask the user to enter the number of elements n, and then allocate memory dynamically for n integers using `malloc`. After allocation, the program should take input for all elements and store them in the allocated array. Once the data is stored, the program should perform basic operations including displaying all elements, finding the maximum element in the list, and searching for a user-specified number (printing its position if found, or "Not found" otherwise). The implementation must strictly use dynamic memory allocation functions (`malloc` and `free`), and should not use static arrays or any form of 2D arrays.

Example:

```
Enter number of elements: 5
Enter elements: 3 8 2 7 5

Elements: 3 8 2 7 5
Maximum element: 8

Enter number to search: 2
Found at position 3
```

Question-2 (20 points)

A small event venue has a row of n seats, where each seat can either be `empty (0)` or `booked (1)`. You are required to write a C program that dynamically allocates memory to represent these seats. Initially, all seats are empty. The program should allow the user to perform basic operations such as viewing seat status, booking a seat, cancelling a booking, and finding the first available seat.

The program must take the number of seats as input and allocate memory using `malloc`. Users can then choose operations from a menu. While performing operations, the program should carefully handle invalid inputs, such as selecting a seat number outside the valid range, trying to book an already booked seat, or cancelling an already empty seat. The system should also be able to report if no seats are available.

Example:

```
Enter number of seats: 5
Menu:
1. Display seats
2. Book a seat
3. Cancel booking
4. First available seat
5. Exit

Enter choice: 1
Seats: 0 0 0 0 0

Enter choice: 2
Enter seat number: 2
Booking successful

Enter choice: 2
Enter seat number: 2
Seat already booked

Enter choice: 4
First available seat: 1

Enter choice: 3
Enter seat number: 2
Cancellation successful

Enter choice: 4
First available seat: 1
```

Question-3 (20 points)

Write a C program to manage student records using structures. Each student record should contain the following fields: Roll Number (integer), Name (string), and Marks (float). The program should first ask the user to enter the number of students n (with a reasonable upper limit), and then store the details of these students using a one-dimensional static array of structures. While taking input, the program should be able to handle names that may contain spaces.

After storing the data, the program should display all student records and determine the student with the highest marks (topper). It should then allow the user to update the marks of a student based on their roll number. If the roll number does not exist, an appropriate message should be displayed.

Finally, the program should ask the user to enter a marks threshold and display the names of all students who have marks greater than this value. If no such students exist, the program should indicate that no records match the condition. The implementation must use structures and a static array only, and should not use dynamic memory allocation or any form of 2D arrays.

Example:

```
Enter number of students: 2

Enter details for student 1:
Roll: 10
Name: Rahul Das
Marks: 65

Enter details for student 2:
Roll: 20
Name: Priya Sen
Marks: 70

All Students:
Roll: 10, Name: Rahul Das, Marks: 65.00
Roll: 20, Name: Priya Sen, Marks: 70.00

Topper: Priya Sen (70.00)

Enter roll to update: 15
Student not found

Enter threshold: 80
Students scoring above 80:
No students found
```